Binomial Heaps

An Efficient Data Structure for Merge-Heavy Systems

Muttakin Ahmed Md. Sium Md. Fuad Al Alam

April 7, 2026

CSE200, Bangladesh University of Engineering and Technology

Part 1: Motivation & Basics

(Presented by Muttakin Ahmed)

The Mailing Service Problem

- **Scenario:** Imagine we are architecting a mailing service.
- **Function:** We receive mail (requests) from users and process them.
- **Constraint:** Our system can only handle one request at a time.
- **The Issue:** When simultaneous requests flood the server, how do we decide which one gets served first?

The Easiest Solution

First-In, First-Out (FIFO)

The easiest way is a standard queue. Whoever sends a message first gets served first.

Why this fails us:

- Not all users are equal!
- What about our *premium subscribers*?
- A simple queue cannot prioritize urgent traffic over regular traffic.

The Priority Queue

Priority Queue Operations

1. Insert(Request)
2. Extract-Max()
   / Extract-Min()

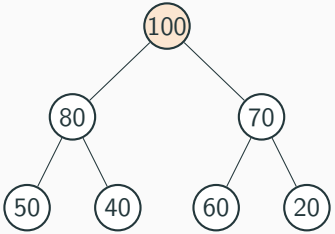
- We need a data structure that always serves the highest priority item first, regardless of arrival time.
- This structure is called a **Priority Queue**.

Implementing a Priority Queue: Binary Heaps

What is a Binary Heap?

- A *perfect* or *almost complete* binary tree.
- It must satisfy the **Heap Property**.
- **Max-Heap:** Every parent is strictly $\geq$ its children.
- **Min-Heap:** Every parent is strictly $\leq$ its children.

Visualizing a Binary Heap



A simple Max-Heap giving priority to the highest value.

Binary Heap Performance

Using a Binary Heap is very efficient for basic tasks!

- **Add (Insert):** $\mathcal{O}(\log n)$ time.
- **Remove (Extract):** $\mathcal{O}(\log n)$ time.

It perfectly serves our basic mailing system!

The Problem of Scale

Service Expansion

Our mailing service becomes wildly popular! We upgrade the servers to use **Multithreading**.

- Each thread now handles a chunk of traffic and builds its own separate heap.
- When threads complete their chunk, their heaps must be **merged** into the unified master heap.

The Merging Bottleneck

The problem with Binary Heaps:

- Merging two binary heaps is **extremely slow**.
- Adding elements one by one takes $\mathcal{O}(n \log n)$ time.
- Creating a brand new array and rebuilding from scratch takes $\mathcal{O}(n)$ time.

Can we merge faster than $\mathcal{O}(n)$?

The Mathematical Analogy: Sticks

Imagine you are physically merging a pile of **25** sticks with **18** sticks.

- If you add them one-by-one to the new pile, it takes you exactly **18 operations**.
- This is strictly linear time $\mathcal{O}(n)$.
- Is there a smarter way to group things?

The Binary Addition Analogy

Instead of counting by 1, consider grouping items in powers of two, just like binary numbers!

- $25_{10} = \mathbf{11001_2}$
- $18_{10} = \mathbf{10010_2}$
- By adding bit-by-bit mathematically, it only takes $\approx$ **5 steps**.
- Each "bit" represents a different magnitude of numbers! (1, 2, 4, 8...).

Logarithmic time $\mathcal{O}(\log n)$.

The Solution: Binomial Heaps

What if our Data Structure mirrored the Binary Number System?

- Instead of one monolithic tree, we use a **forest** (a collection) of structured trees.
- If our trees represented powers of 2, we could literally merge heaps by "adding bits".
- This is exactly what a **Binomial Heap** is!

Part 2: Binomial Trees & Merging

(Presented by Md. Sium)

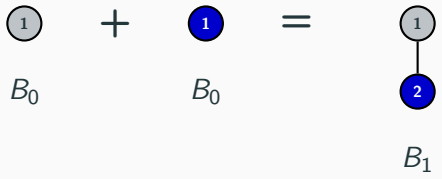
The Building Blocks: Powers of Two

A Binomial Heap is composed of special trees called **Binomial Trees** (B_k).

- Every tree has exactly 2^k nodes.
- We build larger trees by simply snapping two smaller, identical trees together!

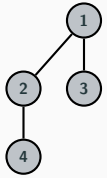
Constructing Binomial Trees: B_0 and B_1

- A B_0 tree has $2^0 = 1$ node.
- A B_1 tree is made by making one B_0 tree the child of another's root. ($2^1 = 2$ nodes).

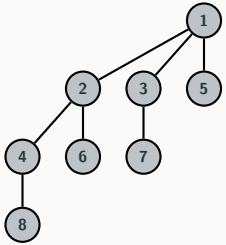


Constructing Binomial Trees: B_2 and B_3

- A B_k tree is always created by joining a B_{k-1} tree with another B_{k-1} tree.
- *Note: The smaller root value always stays on top to preserve the min-heap property!*



B_2 ($2^2 = 4$ nodes)

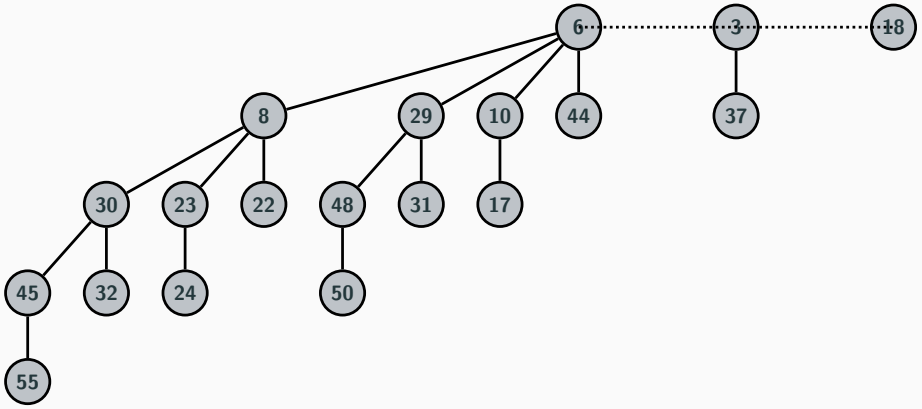


B_3 ($2^3 = 8$ nodes)

The Structure of a Binomial Forest

- Standard properties of a B_k tree:
- It has depth exactly k.
- The root has exactly k children.
- The children are roots of binomial trees of orders $k-1, k-2, \ldots, 0$ ordered from left to right!

Anatomy of a Binomial Heap



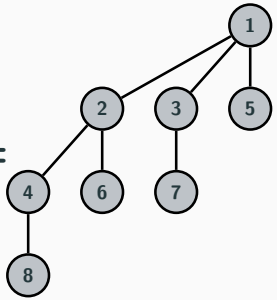
B_4 B_1 B_0

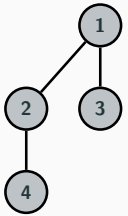
Representing Arbitrary numbers of nodes

If Binomial Trees must have exactly 2^k nodes, how do we store an odd number of requests?

- Suppose our heap has exactly **13 nodes**.
- 13 in binary is $\mathbf{1101_2}$.
- That breaks down to $13 = 8 + 4 + 1$.
- Therefore, our forest must be made up of exactly: $B_3, B_2,$ and B_0.

The 13 Nodes Visualized

$13 =$ 

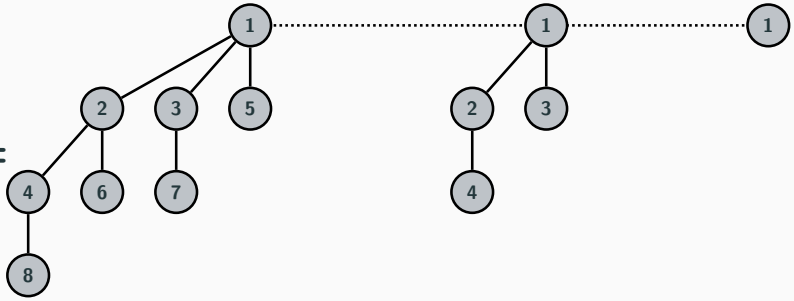
$+$ 

$+$ 

B_3 (8 nodes) B_2 (4 nodes) B_0 (1 node)

The 13 Nodes Visualized

$13 =$



B_3 (8 nodes) B_2 (4 nodes) B_0 (1 node)

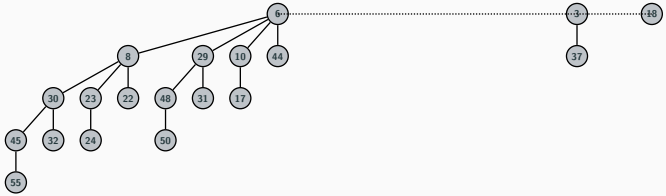
Addition Example: 19 Nodes + 7 Nodes

What if we want to merge a heap containing exactly 7 nodes with a heap containing exactly 19 nodes?

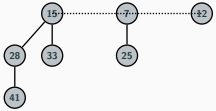
- 19 in binary is $\mathbf{10011_2}$.
- That breaks down to $19 = 16 + 2 + 1$.
- The forest has the trees: $B_4, B_1,$ and B_0.

- 7 in binary is $\mathbf{0111_2}$.
- That breaks down to $7 = 4 + 2 + 1$.
- The forest has the trees: $B_2, B_1,$ and B_0.

Binomial Heap: Union (Initial State)

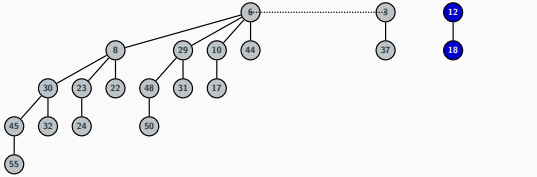


+

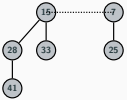


	B_4	B_3	B_2	B_1	B_0	
19	1	0	0	1	1	(10011_2)
+ 7	0	0	1	1	1	(00111_2)

Binomial Heap: Union (Merge B_0)

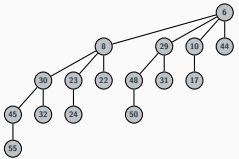


+



	B_4	B_3	B_2	B_1	B_0
19	1	0	0	1	**1**
+ 7	0	0	1	1	**1**
Carry				1	
Result					**0**

Binomial Heap: Union (Merge B_1)



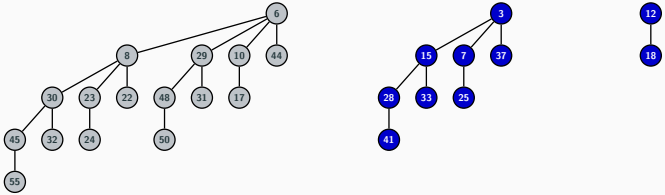
+





	B_4	B_3	B_2	B_1	B_0
19	1	0	0	**1**	1
$+\ 7$	0	0	1	**1**	1
Carry			1	**1**	
Result				**1**	0

Binomial Heap: Union (Merge B_2)

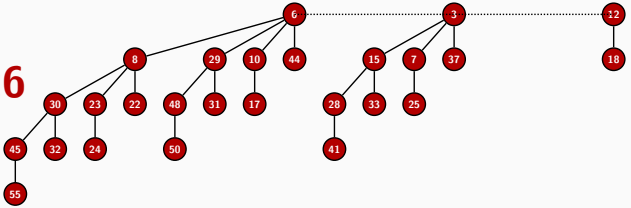


+

	B_4	B_3	B_2	B_1	B_0
19	1	0	**0**	1	1
+ 7	0	0	**1**	1	1
Carry		1	**1**	1	
Result			**0**	1	0

Binomial Heap: Union (Final Result)

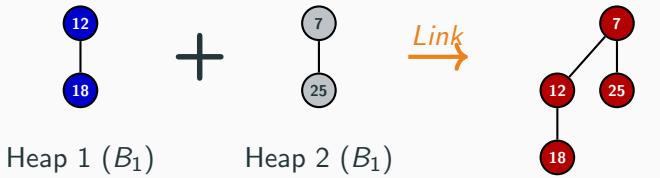
$= 26$



	B_4	B_3	B_2	B_1	B_0	
19	1	0	0	1	1	(10011_2)
$+\ 7$	0	0	1	1	1	(00111_2)
Result	**1**	**1**	**0**	**1**	**0**	$(11010_2 = \mathbf{26})$

What if there ARE Carries?

Let's look at merging two trees of the **same order**.



- We compare the roots: $7 < 12$.
- The 12 (and its subtree) becomes the leftmost child of the 7.
- $B_1 + B_1 =$ Carry B_2!

Merging: Summarized with Carries

If a collision occurs ($1 + 1$ in binary):

- Keep the smaller root.
- Make the larger root its leftmost child.
- The order of the tree increases by 1 (We carry the bit to the left!).
- We repeat this step until there are no remaining duplicate tree orders.

The Magic of Binomial Merging

Because we only traverse up to $\log n$ trees, a full merge operation takes at most $\mathcal{O}(\log n)$ steps. A massive upgrade over $O(n)$!

Part 3: Operations & Complexity

(Presented by Md. Fuad Al Alam)

The Power of Merge

The golden rule of Binomial Heaps:

- The entire data structure revolves around the $O(\log n)$ *Merge* operation!
- For nearly every major operation aside from finding a node, we just configure the data slightly, and then forcefully call *Merge* to handle the heavy lifting.

Operation: Find Minimum

- The smallest value in any B_k tree is strictly its root node.
- Therefore, the smallest value in the entire forest MUST be one of the root nodes.
- Since there are at most $\log n$ trees, we only need to scan the linked list of roots!

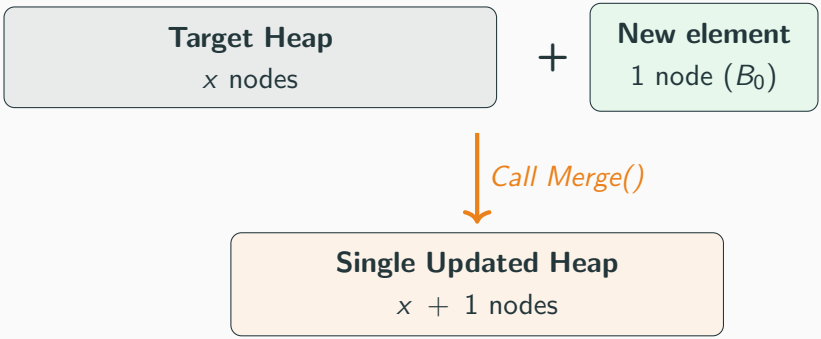
Find Minimum: Complexity Analysis

- Scanning a linked list of size $\log n$ takes $\mathcal{O}(\log \mathbf{n})$.
- **Optimization:** We usually keep an external pointer directly targeting the global minimum root.
- By updating this pointer during merges and extractions, *Find Minimum* becomes a true $\mathcal{O}(\mathbf{1})$ operation.

Operation: Add (Insert)

- Adding a singular request to our mailing priority queue is surprisingly simple!
- We treat the new element not as a standalone item, but as a *brand new Binomial Heap* containing exactly one node (B_0).

Operation: Add (Visualizing Insertion)



Time Complexity: Because *Merge* runs in $O(\log n)$ time, our *Add* operation naturally runs in bounded $\mathcal{O}(\log \mathbf{n})$ time!

Operation: Decrease Key

- Sometimes a premium subscriber upgrades their VIP tier, and their queue priority strictly decreases (gets closer to priority index 0!).
- We adjust the node's value.
- But wait, making a value smaller might violate the min-heap property of its tree!

Operation: Decrease Key (Bubbling)

- We push the decreased value upward toward the root, swapping with its parent until the heap property is restored.
- This is affectionately known as *"Bubbling up"*.
- Because the absolute maximum depth of a tree containing n elements is $\log n$, we can bubble up at most $\log n$ times!
- **Time Complexity:** $\mathcal{O}(\log \mathbf{n})$

Operation: Extract Minimum

- To process and remove the highest priority mail, we must extract the global minimum.
- Step 1: Remove the minimum root from the linked list of roots.
- Step 2: What happens to its orphaned children?

Operation: Extract Minimum (Merging back)

The magic of the Binomial Tree

If you delete the root of a B_k tree, its remaining children are exactly the trees $B_{k-1}, B_{k-2}, \dots B_0$.

- The orphaned children literally form a perfectly valid, standalone Binomial forest!
- We treat the children as a second heap, and call *Merge*() to fuse it back into the main forest.
- **Time Complexity:** $\mathcal{O}(\log \mathbf{n})$

Complexity Summary: The Binomial Advantage

Everything piggybacks on the efficient Merge!

Operation	**Binomial Heap**
Find Minimum	$\mathcal{O}(1)$ *(with pointer)*
Insert Node	$\mathcal{O}(\log n)$
Decrease Key	$\mathcal{O}(\log n)$
Merge Heaps	$\mathcal{O}(\log n)$ $\leftarrow$ The Engine!
Extract Minimum	$\mathcal{O}(\log n)$

Comparison: Binary vs. Binomial Heaps

Operation	Binary Heap	Binomial Heap
Find Min	$\mathcal{O}(1)$	$\mathcal{O}(1)$
Insert	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
Extract Min	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
Merge	$\mathcal{O}(n)$	$\mathcal{O}(\log \mathbf{n})$

Takeaway: For multithreaded or distributed systems where heaps must coalesce, Binomial Heaps reign supreme.

The Theoretical Ideal: Fibonacci Heaps

Could we push the limits even further? Yes, using **Fibonacci Heaps**.

- Employs severe *"lazy evaluation"*.
- It simply throws trees together in $O(1)$ time and delays actually merging them until an *Extract* call absolutely forces it to do the work.
- Achieves an unheard of $\mathcal{O}(1)$ **amortized time** for Insert, Merge, and Decrease Key!

The Catch: Why Binomial wins in Practice

- While Fibonacci Heaps are mathematically beautiful and give Dijkstra's algorithm its theoretical fastest bounds...
- They suffer from immense **constant overhead factors**.
- They rely heavily on pointers, causing massive CPU cache misses.
- Their implementation is exceptionally complex and prone to errors.

Conclusion

Binomial Heaps strike the perfect, practical balance between optimal merge performance and sensible engineering complexity.

Thank You!